

Sure. Here is **Set 7 — 50 more scenario-based GenAI interview questions with practical answers**. These are closer to what a **4+ years experienced candidate** may face in a technical/managerial interview.

■ GenAI 4+ Years Interview — Set 7

301. Your chatbot gives different answers to the same question. Why?

Answer:

"LLM generation can be non-deterministic, especially with higher temperature. I would reduce temperature when deterministic responses are required and ensure the retrieved context and prompt are consistent."

302. How do you make LLM responses more deterministic?

Answer:

"I would use a low or zero temperature where supported, control the prompt, use consistent retrieval, and validate the output against expected rules."

303. Your answer is sometimes correct and sometimes hallucinated. What do you check?

Answer:

"I would check retrieval consistency, similarity threshold, context quality, prompt instructions, temperature, and whether the required information actually exists in the retrieved context."

304. What if temperature is already 0 but hallucination still happens?

Answer:

"Temperature alone doesn't prevent hallucination. I would improve retrieval, enforce context-grounded prompting, add a confidence threshold, validate the output, and explicitly instruct the model to say 'I don't know' when the context is insufficient."

■ Chunking Scenarios

305. Your chunks are too small. What problem can happen?

Answer:

"Important context may be separated across multiple chunks. Retrieval might find one small piece without the surrounding information required to answer the question."

306. Your chunks are too large. What problem can happen?

Answer:

"Chunks may contain multiple unrelated topics. Retrieval becomes less precise and more irrelevant context is sent to the LLM, increasing latency and cost."

307. How do you choose chunk size?

Answer:

"I would consider document structure, sentence boundaries, token limits, query types, and retrieval evaluation. I would test different chunk sizes rather than choosing one value universally."

308. Why use chunk overlap?

Answer:

"Overlap preserves information that lies at the boundary between chunks, reducing the chance that an important sentence is split away from its context."

309. Too much overlap is causing high storage cost. What do you do?

Answer:

"I would reduce overlap and evaluate retrieval recall. Overlap should be enough to preserve context, not unnecessarily duplicate large amounts of text."

■ Ingestion Pipeline

310. A user uploads the same PDF five times. What happens?

Answer:

"I would use a document hash or unique document ID to detect duplicates. If the content hasn't changed, I would avoid regenerating embeddings unnecessarily."

311. Why use a content hash?

Answer:

"A content hash provides a compact fingerprint of the document. If the hash is unchanged, the content is unchanged and reprocessing can be skipped."

312. What if only one page of a 500-page PDF changes?

Answer:

"If the document-processing architecture supports it, I would identify the changed portion and reprocess only the affected chunks. Otherwise, I would version the document and rebuild the necessary index carefully."

313. An ingestion job succeeds but some chunks are missing. How do you detect this?

Answer:

"I would maintain ingestion metadata such as document ID, expected chunk count, processed chunk count, and status. After processing, I would validate that all expected chunks were indexed."

314. How do you make ingestion idempotent?

Answer:

"The same ingestion request should produce the same final state without creating duplicates. I would use unique document/chunk IDs and check existing versions before inserting."

■ Vector Database Scenarios

315. Vector DB contains 10 million vectors and search is slow. What do you investigate?

Answer:

"I would check index configuration, vector dimensions, filtering strategy, hardware, database load, query complexity, and whether the selected similarity index is appropriate for the workload."

316. Why is vector dimension important?

Answer:

"Higher-dimensional vectors can increase storage and computational requirements. The dimension is determined by the embedding model, so changing embedding models may require rebuilding the vector index."

317. Can you mix embeddings from different models in the same vector index?

Answer:

"Generally, no. Different embedding models may produce incompatible vector spaces and dimensions. I would keep separate indexes or rebuild the index using one compatible embedding model."

318. What happens when you change the embedding model?

Answer:

"Existing vectors need to be regenerated using the new embedding model because the old and new vectors may not be directly comparable."

■ Retrieval Scenarios

319. Top-K = 3 works for some questions but not others. What do you do?

Answer:

"I would consider dynamic Top-K based on similarity scores or query complexity. I would also use reranking and evaluate different retrieval configurations on representative queries."

320. Why shouldn't you always use Top-K = 10?

Answer:

"It may retrieve unnecessary information, increase token usage and latency, and potentially distract the LLM with irrelevant context."

321. Why shouldn't you always use Top-K = 1?

Answer:

"The first result may not contain all required information. Complex questions may require multiple chunks or documents."

322. Similarity threshold is too high. What happens?

Answer:

"Relevant documents may be rejected, causing false negatives and 'I don't know' responses."

323. Similarity threshold is too low. What happens?

Answer:

"Irrelevant chunks may pass retrieval and reach the LLM, increasing the risk of poor answers and hallucination."

■ Reranking Scenarios

324. Your first vector result is irrelevant but the fourth result is highly relevant. What does that tell you?

Answer:

"The initial retrieval ranking isn't optimal. I would evaluate the embedding model, query formulation, and consider reranking the candidate set."

325. What is a cross-encoder reranker?

Answer:

"A cross-encoder evaluates the query and candidate document together and produces a relevance score. It can provide more accurate ranking than basic vector similarity but usually requires more computation."

326. Would you rerank 10,000 documents?

Answer:

"Usually no. I would first use efficient retrieval to reduce the candidate set and then rerank a much smaller number of candidates."

■ RAG Evaluation

327. How do you measure retrieval quality?

Answer:

"I would use metrics such as precision, recall, hit rate, MRR, or nDCG depending on the evaluation design."

328. What is MRR?

Answer:

"Mean Reciprocal Rank measures how highly the first relevant result appears in the retrieved ranking. Higher MRR means relevant results tend to appear closer to the top."

329. What is hit rate?

Answer:

"Hit rate measures how often at least one relevant result appears within the retrieved Top-K results."

330. Your retrieval metrics are excellent but users still dislike answers. Why?

Answer:

"Retrieval quality is only one part of RAG. The problem could be prompt design, answer generation, context ordering, verbosity, missing information, or poor response formatting."

■ Answer Evaluation

331. How do you measure answer quality?

Answer:

"I would evaluate correctness, relevance, completeness, groundedness, and possibly citation accuracy using a combination of automated evaluation and human review."

332. What is groundedness?

Answer:

"Groundedness measures whether the generated answer is supported by the provided source context."

333. The answer is factually correct but not supported by retrieved documents. Is that acceptable?

Answer:

"For a strict enterprise RAG application, I would consider it a grounding issue. The answer should be supported by the authorized knowledge base rather than relying on unsupported model knowledge."

■ Prompt Engineering

334. What is a good RAG system prompt?

Answer:

A simple approach is:

You are an enterprise knowledge assistant.

Answer only using the provided context.

If the context does not contain enough information, say:

"I don't know based on the provided documents."

Do not invent facts.

Provide the source when available.

335. Why explicitly say "I don't know"?

Answer:

"It establishes a clear fallback behavior when retrieval doesn't provide sufficient evidence and helps reduce unsupported answers."

336. Your prompt says "answer only from context," but hallucinations still occur. What next?

Answer:

"Prompt instructions alone aren't sufficient. I would improve retrieval, enforce similarity thresholds, reduce irrelevant context, validate generated answers, and evaluate groundedness."

■ Output Validation

337. Why validate LLM output?

Answer:

"LLMs generate probabilistic text. Output validation can enforce required formats, detect prohibited information, verify fields, and identify unsupported responses."

338. Your application requires JSON output but the LLM returns invalid JSON. What do you do?

Answer:

"I would use structured output or a schema-based response mechanism where supported, validate the response, and retry or reject invalid output when necessary."

339. What if the model returns extra text before the JSON?

Answer:

"I would enforce structured output at the model/API level where possible rather than relying only on string parsing. I would then validate against the expected schema."

■ Production API

340. Your API receives 10,000 requests per minute. What do you do?

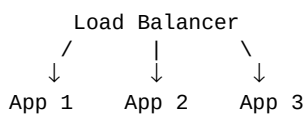
Answer:

"I would use load balancing, horizontal scaling, rate limiting, caching, connection pooling, asynchronous processing where appropriate, and monitor CPU, memory, latency, error rates, and downstream API limits."

341. What is horizontal scaling?

Answer:

"Adding more application instances to distribute traffic."



342. What is auto-scaling?

Answer:

"Auto-scaling automatically increases or decreases application instances based on workload or metrics such as CPU, request count, queue depth, or latency."

343. Why use a load balancer?

Answer:

"It distributes incoming requests across multiple application instances, improving scalability and availability."

■ Rate Limiting

344. One user sends 1,000 requests per second. What do you do?

Answer:

"I would use rate limiting per user, API key, or tenant. I would return a controlled rate-limit response and monitor suspicious usage."

345. Why is rate limiting important for GenAI?

Answer:

"LLM requests can be expensive. Rate limiting protects the system from abuse, prevents excessive cost, and protects downstream APIs from overload."

■ Failure Scenarios

346. Vector DB is healthy but embeddings API is down. What happens?

Answer:

"New queries requiring embeddings may fail, even though the vector database is available. I would use monitoring, retries for transient failures, and potentially a fallback embedding service if the architecture supports it."

347. LLM is down but vector search works. Can you still answer?

Answer:

"Not normally through the LLM generation pipeline. The system could expose retrieved sources or use a fallback model if configured, but it should not invent an answer."

348. Vector DB is down but LLM is working. Should the LLM answer?

Answer:

"For enterprise knowledge questions, I would generally return a controlled error or fallback rather than relying on the LLM's pretrained knowledge, because the current authorized documents aren't available."

■ Model Routing

349. Would you use the same LLM for every request?

Answer:

"Not necessarily. I could route simple tasks to smaller, faster models and complex reasoning tasks to more capable models based on business requirements."

350. What is model routing?

Answer:

"Model routing selects an appropriate model based on the request type, complexity, cost, latency, or other policies."

■ ■ Very Important Scenario

351. Interviewer: "Your RAG system has good retrieval accuracy, but users complain that answers are too long and contain unnecessary information. How would you fix it?"

Answer:

"Since retrieval accuracy is already good, I would focus on the generation layer. I would update the prompt to request concise, direct answers, reduce the amount of context passed to the model where possible, set an appropriate output-token limit, and define a clear response format. I would then evaluate answer length and relevance against a test dataset."

Simple interview answer:

"I would not immediately change retrieval because retrieval is already good. I would improve the prompt, reduce unnecessary context, control output tokens, and evaluate the response quality."

352. Interviewer: "Your RAG retrieves irrelevant chunks. What would you do?"

Answer:

"I would check the chunking strategy and embedding model first. Then I would tune Top-K and similarity threshold, add metadata filtering, and introduce reranking. Finally, I would measure retrieval precision and recall."

353. Interviewer: "Your RAG misses relevant chunks. What would you do?"

Answer:

"I would investigate recall. I would review chunk size and overlap, embedding quality, query rewriting, Top-K, and hybrid search. I would then compare retrieval recall using a fixed evaluation dataset."

354. Interviewer: "Your RAG is accurate but expensive."

Answer:

"I would reduce unnecessary context and output tokens, optimize Top-K, use reranking to select only relevant chunks, introduce caching, and consider a smaller model for simpler requests. I would monitor token usage and cost per request."

355. Interviewer: "Your RAG is accurate but slow."

Answer:

"I would measure latency across embedding, retrieval, reranking, LLM generation, and network calls. Then I would optimize the slowest component using caching, smaller context, faster models, parallel processing, or infrastructure scaling."

■ 356. Interviewer: "Documents are frequently changing. RAG or fine-tuning?"

Answer:

"I would choose RAG because the knowledge is frequently changing. I can update the documents, regenerate embeddings, and update the vector database without retraining the LLM. Fine-tuning is more appropriate when I need to change the model's behavior, style, or task performance."

■ 357. Interviewer: "How do you prevent hallucination?"

Answer:

"I would use high-quality retrieval, similarity thresholds, reranking, a strong grounded prompt, limited context, and an explicit 'I don't know' fallback. I would also evaluate groundedness and answer correctness."

■ 358. Interviewer: "How do you handle irrelevant questions?"

Answer:

"I would use intent detection or a relevance check before retrieval and a similarity threshold after retrieval. If the question is outside the knowledge base or no chunks are sufficiently relevant, I would return a controlled response."

■ 359. Interviewer: "How do you handle thousands of PDFs?"

Answer:

"I would use object storage for documents, asynchronous ingestion through a queue, parallel workers for extraction and embedding, batch processing, a scalable vector database, metadata filtering, monitoring, retries, and auto-scaling."

■ 360. Interviewer: "What is your approach when you don't know the answer?"

Answer:

"I don't guess. First I check whether the required information exists in the retrieved context. If the similarity score is below the configured threshold or the context doesn't contain enough information, I return 'I don't know based on the available documents.' "

■ 4+ Years Interview Golden Formula

For scenario questions, structure your answer like this:

****Problem → Check → Fix → Validate****

Example:

Problem:

"Users are getting irrelevant answers."

Check:

"I would check retrieved chunks, similarity scores, chunking, embeddings, Top-K, and metadata filters."

Fix:

"I would improve chunking, use a threshold, reranking, and metadata filtering."

Validate:

"I would measure retrieval precision/recall and answer groundedness."

Memorize this:

"First I identify the failing component, then I check the metrics and logs, apply a targeted fix, and validate the improvement using an evaluation dataset and production monitoring."

That answer pattern is extremely useful when the interviewer gives you a **new scenario you have never seen before**.